CS 6341 Robotics Homework 3

Instructor Yu Xiang

October 8, 2026

In this homework, write down your solutions for problems 1, 2 and finish the coding problem 3. Upload your solutions and code to eLearning. Our TA will check your solutions and run your scripts to verify them.

Problem 1

(2.5 points)

Forward Kinematics. Exercise 4.12 in Lynch and Park, Modern Robotics.

The RRPRRR spatial open chain of Figure 1 is shown in its zero position (all joints lie on the same plane). Determine the end-effector zero position configuration M, the screw axes $\mathcal{S}_i$ in {0}, and the screw axes $\mathcal{B}_i$ in {b}. Setting $\theta_5 = \pi$ and all other joint variables to zero, find T_{06} and T_{60}.

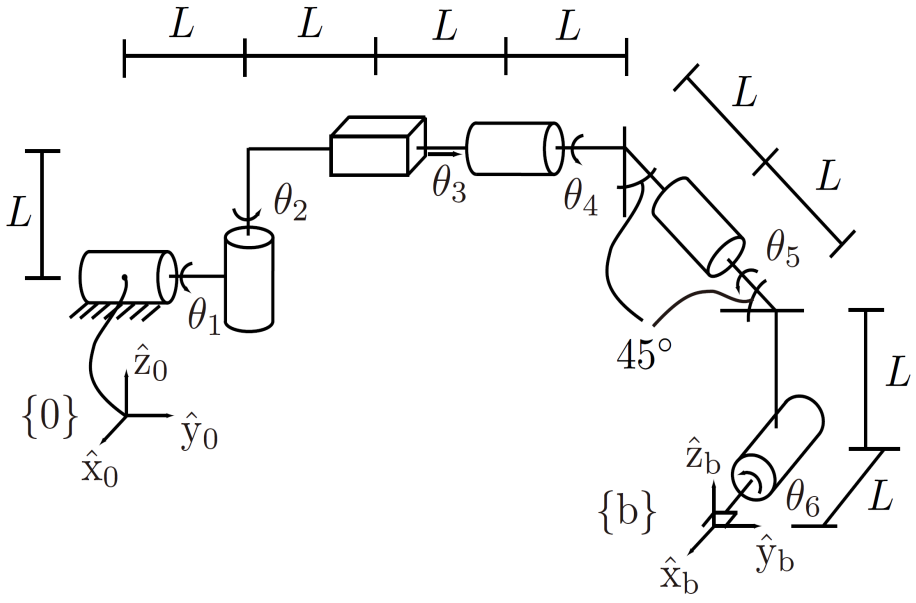


Figure 1: An RRPRRR spatial open chain.

Problem 2

(2.5 points)

Velocity Kinematics. Exercise 5.11(a) in Lynch and Park, Modern Robotics.

The spatial 3R open chain of Figure 2 is shown in its zero position. Let p be the coordinates of the origin of {b} expressed in {s}. In its zero position, suppose we wish to make the end-effector move with linear velocity $\dot{p} = (10, 0, 0)$. What are the required input joint velocities $\dot{\theta}_1$, $\dot{\theta}_2$ and $\dot{\theta}_3$?

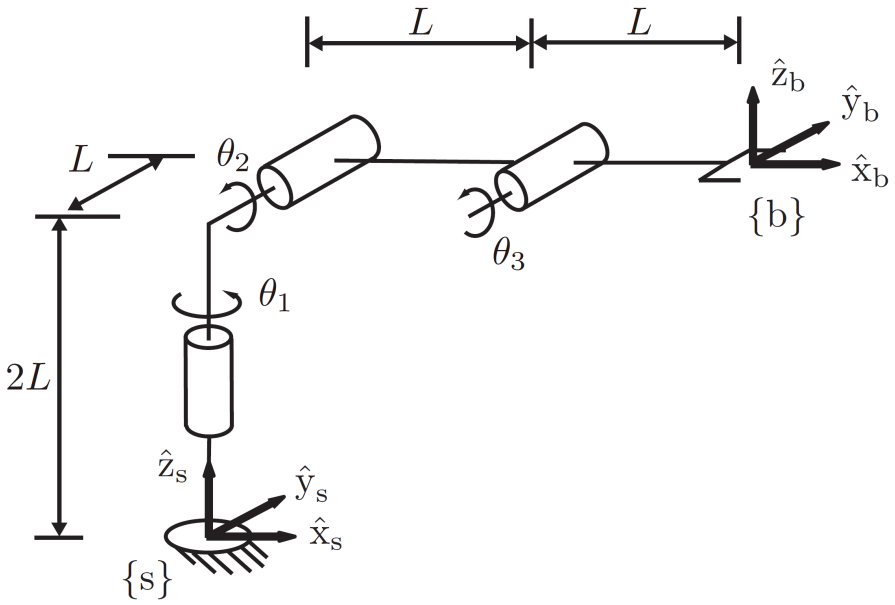


Figure 2: A spatial 3R open chain.

Problem 3

(3 points)

ROS 2 Programming: MoveIt 2 Planning Scene, Forward Kinematics, and Inverse Kinematics.

In this problem, you will use ROS 2 and MoveIt 2 to create a planning scene and compute the forward kinematics (FK) and inverse kinematics (IK) of the Fetch robot.

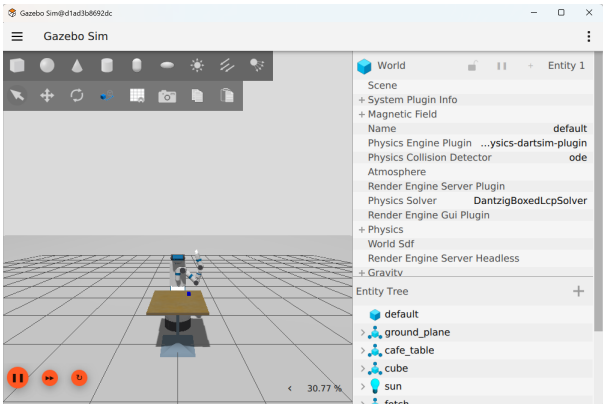
For the following steps, continue using the same ROS 2 Jazzy workspace and software environment from Homework 2.

(3.1) Launch the Fetch Gazebo simulation and MoveIt 2.

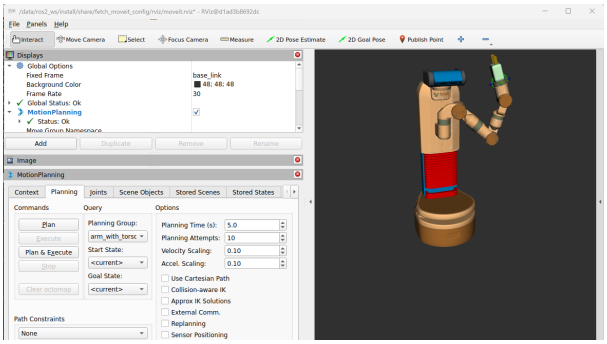
Launch the Fetch Gazebo simulator using the same setup from Homework 2. You can use Terminator to open multiple terminals.

- `ros2 launch panda fetch.launch.py`

You should see both the Gazebo environment and RViz, as shown in Figure 3.



Gazebo simulation



RViz

Figure 3: Gazebo and RViz after launching the Fetch simulation.

Before continuing, make sure that Gazebo, RViz, and MoveIt 2 are running correctly. You can verify that the robot joint states are being published by running:

- `ros2 topic echo /joint_states --once`

(3.2) Install `pymoveit2`.

We will use `pymoveit2`, a Python interface for MoveIt 2, to compute FK and IK.

Clone the source code into the `src` directory of your ROS 2 workspace. Use the `main` branch:

- `git clone https://github.com/IRVLUTD/pymoveit2.git`

Build your ROS 2 workspace again to include this package, and source the workspace after building:

- `cd ~/data/ros2_ws`

- `colcon build`
- `source install/setup.bash`

Make sure that the workspace builds successfully without any errors before continuing.

(3.3) Create a planning scene and compute FK and IK.

Download the planning_scene_block.py file from eLearning.

The provided Python script performs the following steps:

1. Queries the pose of the demo cube in the Gazebo environment using the same approach as in Homework 2.
2. Creates a MoveIt 2 planning scene and adds the cube as a collision object.
3. Reads the current joint configuration of the Fetch robot.
4. Computes the end-effector pose using forward kinematics.
5. Computes an inverse kinematics solution for the same end-effector pose.

Complete all sections marked TODO in the Python script and run your completed program.

After running the script, you should see a green cube in the MoveIt 2 planning scene, as shown in Figure 4.

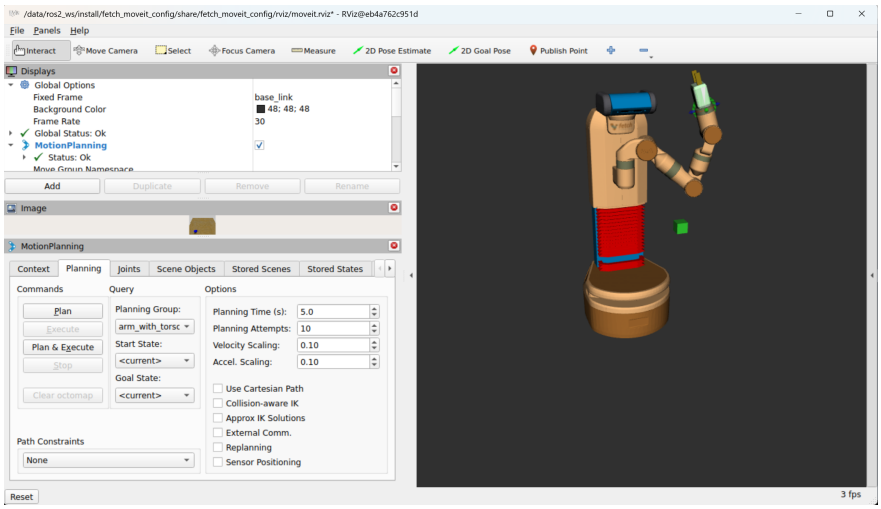


Figure 4: RViz visualization of the MoveIt 2 planning scene with the cube added as a collision object.

Your program should also print the current joint coordinates, the end-effector pose computed by FK, and the joint coordinates computed by IK. An example output is shown in Figure 5.

The computed IK solution should be close to the current joint coordinates of the robot. Small numerical differences are expected. Note that, in general, a robot may have multiple valid IK solutions for the same end-effector pose.

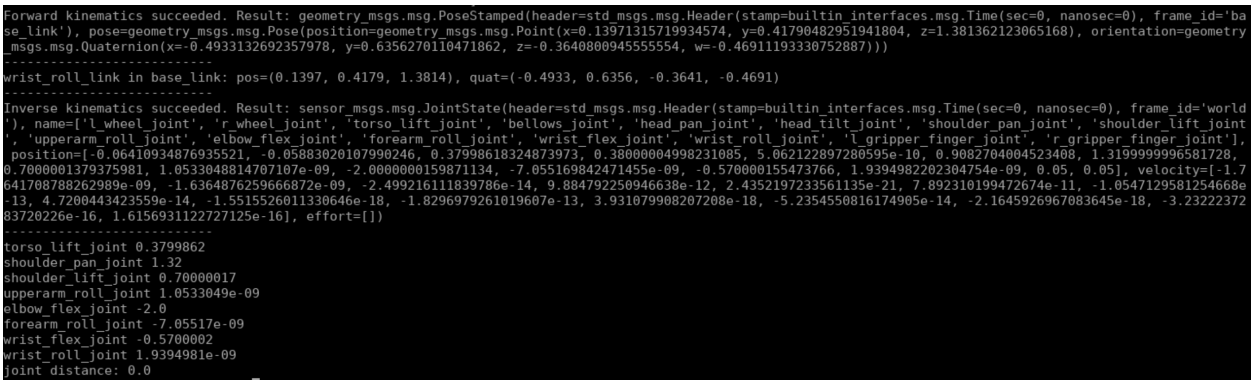


Figure 5: Example output of the FK and IK computations using `pymoveit2`.

Submission guideline: Upload the following to eLearning:

1. Your completed `planning_scene_block.py` file.
2. A screenshot of RViz showing the planning scene with the green cube, similar to Figure 4.
3. A screenshot of the terminal output showing the FK and IK results, similar to Figure 5.

Problem 4

(2 points)

ROS 2 Programming: SO-101 Simulation and MoveIt 2.

In this problem, you will set up the SO-101 robot in ROS 2 Jazzy and use Gazebo, ROS 2 Control, and MoveIt 2 to plan and execute a robot motion.

We will use the following ROS 2 package maintained by our lab:

`https://github.com/IRVLUTD/lerobot_ws`

Use the same ROS 2 Jazzy environment from the previous homeworks.

(4.1) Install the SO-101 ROS 2 package.

Clone the repository:

- `git clone https://github.com/IRVLUTD/lerobot_ws.git`
- `cd lerobot_ws`

Install the required dependencies:

- `rosdep update`
- `rosdep install --from-paths src --ignore-src -r -y`

Build and source the workspace:

- `colcon build`
- `source install/setup.bash`

Make sure that the workspace builds successfully without errors before continuing.

(4.2) Launch the SO-101 simulation and ROS 2 Control.

Open two terminals and source the workspace in both terminals.

In the first terminal, launch the SO-101 Gazebo simulation:

- `ros2 launch lerobot_description so101_gazebo.launch.py`

In the second terminal, launch the ROS 2 controllers:

- `ros2 launch lerobot_controller so101_controller.launch.py`

Verify that the controllers are running:

- `ros2 control list_controllers`

Make sure that the controllers for the SO-101 arm and gripper are active.

(4.3) Plan and execute a motion using MoveIt 2.

Keep Gazebo and the ROS 2 controllers running. Open a third terminal, source the workspace, and launch MoveIt 2:

- `ros2 launch lerobot_moveit so101_moveit.launch.py`

In the MoveIt 2 Motion Planning panel:

1. Select the `arm` planning group.
2. Select `ompl` as the planning library.
3. Set a valid target configuration for the SO-101 arm.
4. Click **Plan** to generate a trajectory.
5. Click **Execute** to execute the planned trajectory.

Observe the planned motion in RViz and verify that the SO-101 arm executes the corresponding motion in Gazebo.

You can also try planning and executing a motion for the gripper.

Submission guideline: Upload the following to eLearning:

1. A screenshot showing the SO-101 robot running in Gazebo.
2. A screenshot showing a successfully planned trajectory for the SO-101 arm in MoveIt 2.
3. A screenshot of the output from `ros2 control list_controllers` showing that the SO-101 controllers are active.